02

Part 2 · Compute & Scaling

How workloads run: virtual machines, containers, serverless, and how capacity grows and shrinks.

- Virtualization · Hypervisor · VM
- Containers vs VMs · Kubernetes
- Serverless
- Scalability vs Elasticity
- Vertical vs Horizontal Scaling
- Auto Scaling

2.1 Virtualization

★★★

IN ONE LINE

Software technology that splits one physical machine into many isolated virtual ones.

Virtualization uses software to turn one physical machine into **several isolated virtual machines**, each behaving like a real computer with its own OS. It is the technology that makes the cloud economical: providers split huge servers among many customers.

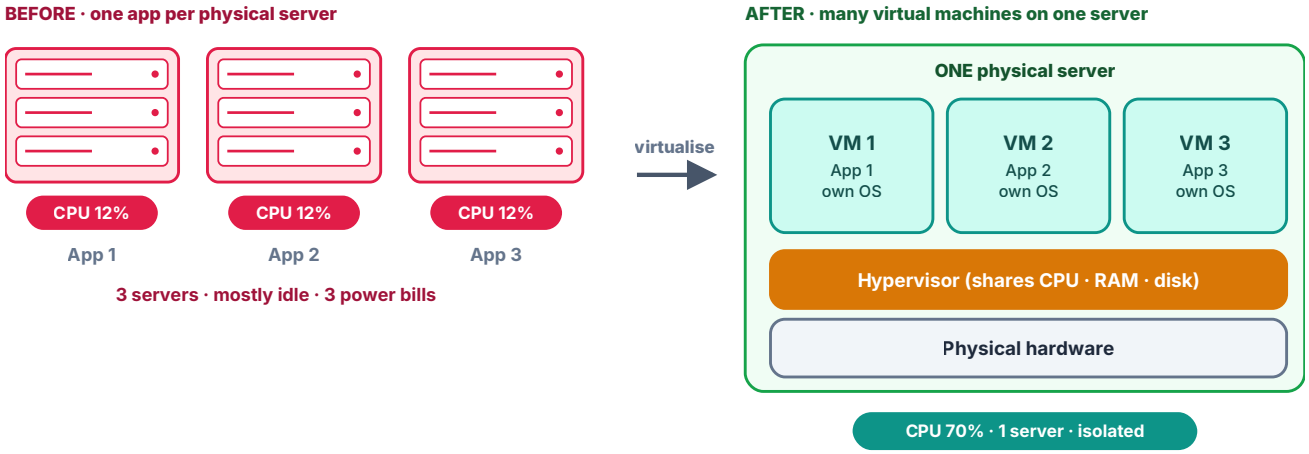


Figure 2.1 — From one-app-per-server to many VMs per server

Benefits

- **Higher utilisation** — less idle hardware.
- **Isolation** — a crash in one VM does not affect another.
- **Speed** — create a VM in seconds from an image.
- **Portability** — move or snapshot a VM.
- **Cost** — fewer physical servers.

Kinds of virtualization

- **Server** virtualisation (VMs) — this topic.
- **Storage** — pool many disks into one logical volume.
- **Network** — virtual networks and switches (VPC, 4.6).
- **Desktop** — remote virtual desktops (VDI).
- **OS-level** — containers (2.4).

REAL-WORLD ANALOGY

An apartment building: one physical building (server) divided into private flats (VMs). Residents share the foundations and plumbing but cannot walk into each other's homes.

2.2 Hypervisor

★★★

IN ONE LINE

The software layer that creates and manages virtual machines on shared hardware.

A **hypervisor** (virtual machine monitor) is the software layer that **creates, runs and isolates virtual machines**, handing each one a share of the physical CPU, memory, storage and network. It is the heart of virtualisation.

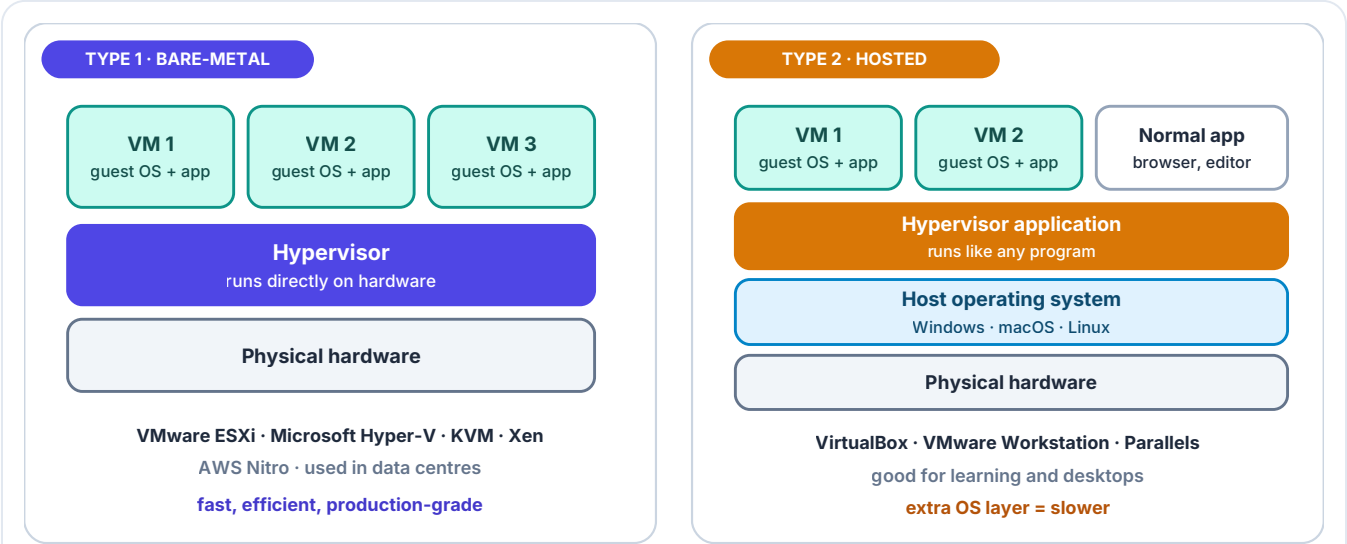


Figure 2.2 — Type 1 (bare-metal) vs Type 2 (hosted) hypervisors

	Type 1 (bare-metal)	Type 2 (hosted)
Runs on	Directly on hardware	On top of a host OS
Performance	High, low overhead	Lower (extra OS layer)
Used in	Cloud and enterprise data centres	Developer laptops, labs
Examples	ESXi, Hyper-V, KVM, Xen	VirtualBox, VMware Workstation

INTERVIEW ANGLE

Public clouds use type 1 hypervisors (AWS uses a KVM-based “Nitro” system, Azure uses Hyper-V, GCP uses KVM). The hypervisor is the part of the **shared responsibility model** that is always the provider’s job. If asked “what stops one customer’s VM from reading another’s memory?” — **hypervisor isolation**.

2.3 Virtual Machine (VM)

★★★

IN ONE LINE

A software-defined computer with its own OS, running on shared physical hardware.

A **virtual machine (VM)** is a software-defined computer: it has virtual CPUs, memory, disk and network, runs its own full operating system, and behaves like a physical server. In the cloud it is called an **instance** (EC2 instance, Azure VM, Compute Engine instance).

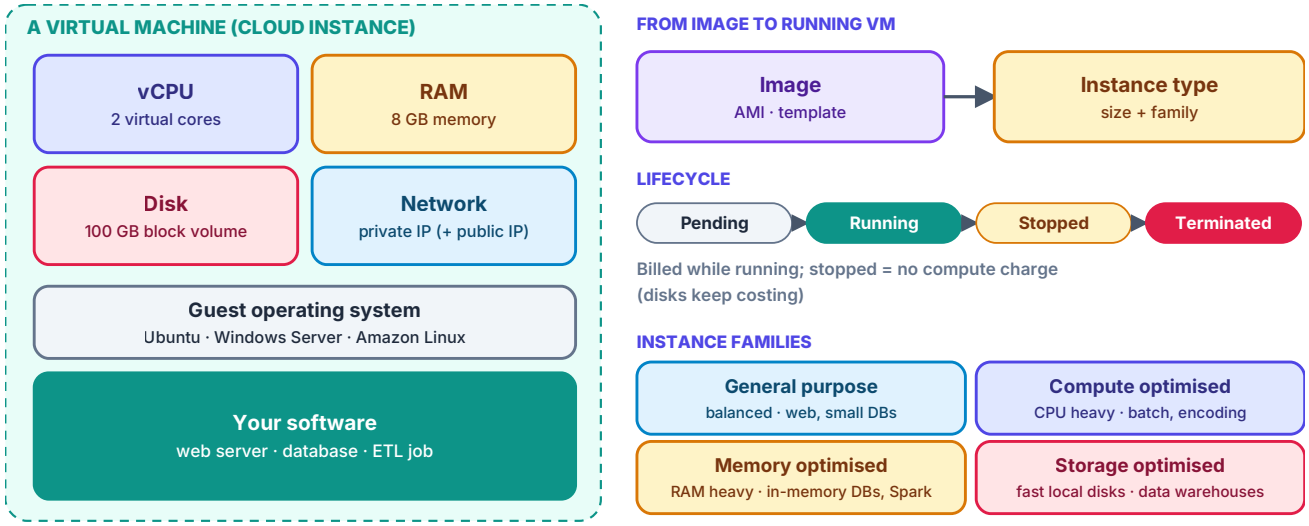


Figure 2.3 — Anatomy, image, lifecycle and families of a cloud VM

Term	Meaning
Image (AMI, VM image)	A template: OS + pre-installed software. Launch many identical VMs from one image.
Instance type	The size/shape: how many vCPUs and how much RAM (e.g. `m5.large`).
Snapshot	A point-in-time copy of a disk, used for backup and cloning.
Key pair / SSH	How you log in to a Linux VM securely, without a password (port 22).
User data / bootstrap	A script that runs on first boot to install and configure software automatically.

DATA ENGINEERING TIE-IN

Data engineers use VMs to host self-managed tools (Airflow, Kafka, Spark clusters) and pick **memory-optimised** instances for Spark and **storage-optimised** ones for database nodes. Managed services remove this VM work — that is the PaaS trade-off.

2.4 Container vs Virtual Machine

★★★

IN ONE LINE

Containers share the host OS and are lightweight; VMs carry a full OS each and isolate more strongly.

Both isolate applications, but at different levels. A VM virtualises the **hardware** (each VM carries a full OS). A container virtualises the **operating system** (all containers share the host OS kernel and package only the app and its libraries).

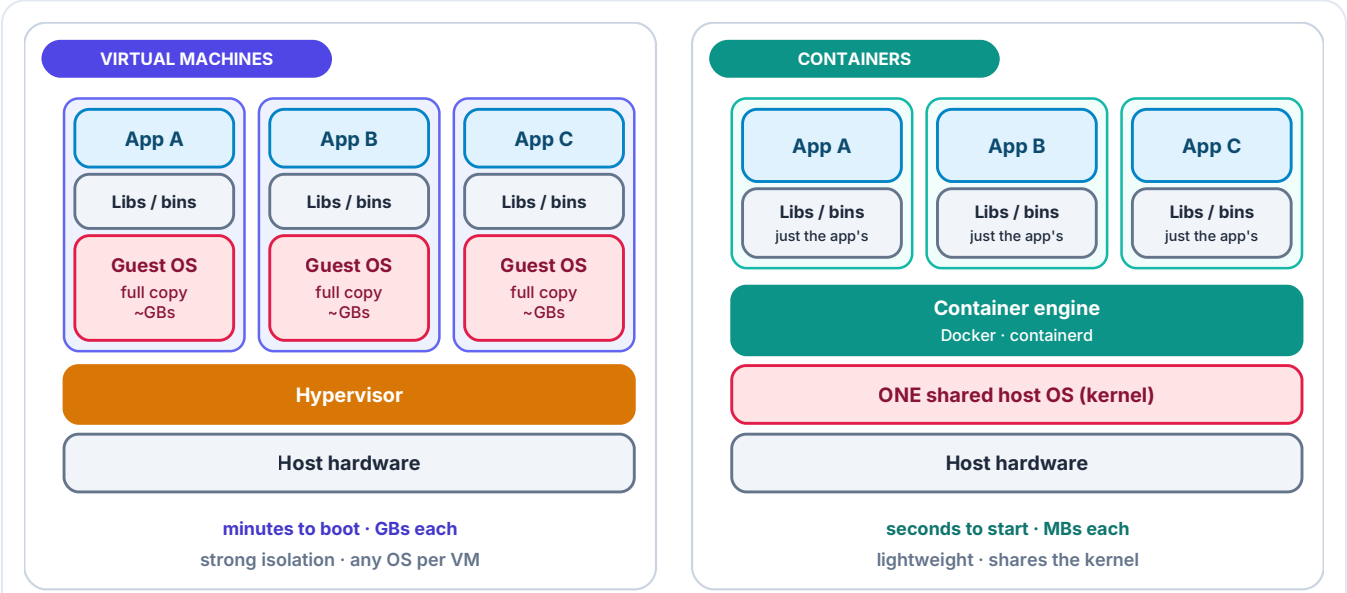


Figure 2.4 — VMs vs containers: why containers are lighter

	Virtual machine	Container
Isolation level	Hardware (strong)	Process / OS (lighter)
Size	GBs (includes OS)	MBs
Start-up time	Minutes	Seconds or less
Density	Tens per server	Hundreds per server
OS flexibility	Any OS per VM	Must match host kernel (Linux on Linux)
Best for	Legacy apps, strong isolation, different OSes	Microservices, CI/CD, portable apps, bursts of workers
Tools	VMware, Hyper-V, EC2	Docker, Podman, Kubernetes

Docker in 30 seconds

Key words

- **Dockerfile** — recipe for building an image.
- **Image** — read-only template (app + dependencies).
- **Container** — a running instance of an image.
- **Registry** — where images live (Docker Hub, ECR, ACR).

DOCKERFILE

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "etl_job.py"]
```

INTERVIEW ANGLE

"It works on my machine" is the classic problem containers solve: the **same image** runs identically on a laptop, in CI and in production. They are not a replacement for VMs — in the cloud, containers usually run *on* VMs.

2.5 Container Orchestration & Kubernetes

★★☆

IN ONE LINE

Automating deployment, scaling and healing of many containers across many machines.

Running one container is easy. Running **hundreds across many machines** — restarting failed ones, scaling, rolling out updates — needs an **orchestrator**. **Kubernetes (K8s)** is the industry standard. Managed versions: **EKS** (AWS), **AKS** (Azure), **GKE** (Google).

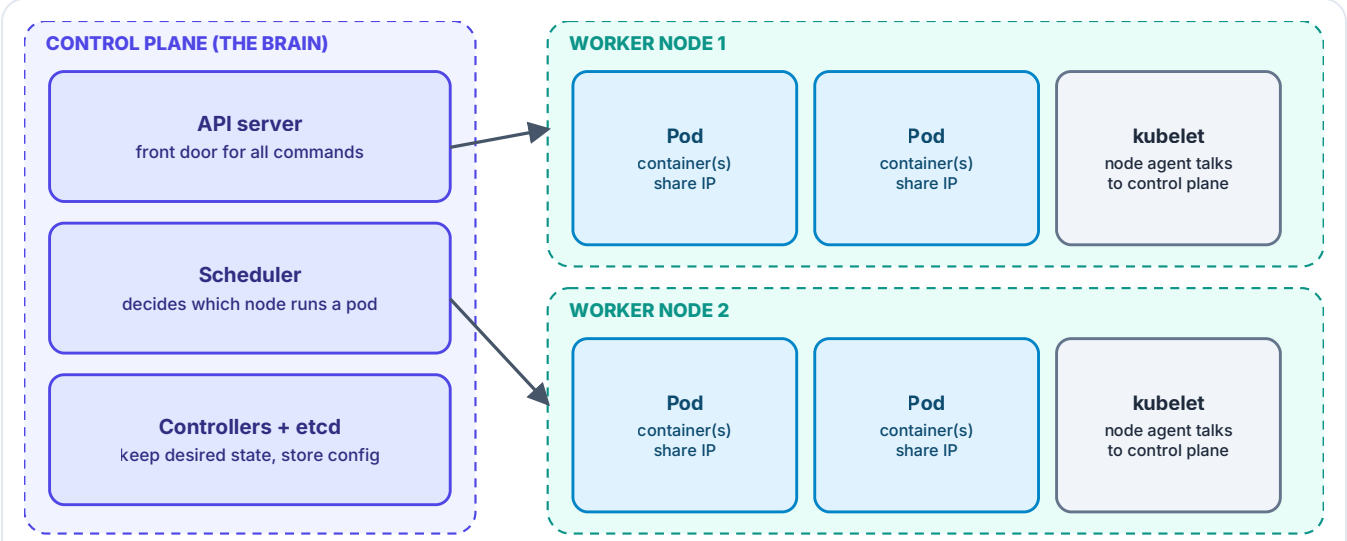


Figure 2.5 — Kubernetes: control plane plus worker nodes running pods

Concept	Meaning
Pod	Smallest deployable unit: one or more containers sharing network and storage.
Node	A worker machine (VM) that runs pods.
Deployment	Declares "keep N replicas of this pod running" and handles rolling updates.
Service	A stable address / load balancer in front of changing pods.
Desired state	You declare what you want; Kubernetes continuously works to make reality match (self-healing).

DATA ENGINEERING TIE-IN

Spark on Kubernetes, Airflow on Kubernetes (KubernetesExecutor) and containerised ETL jobs are common. At fresher level, knowing **what problem Kubernetes solves** matters more than writing manifests.

2.6 Serverless Computing & Serverless Architecture

★★★

IN ONE LINE

Run code in response to events without managing servers; pay only while it executes.

Serverless means you write code and the provider runs it **on demand**: it provisions, scales, patches and bills per execution. You never see or manage the servers (they exist, but they are the provider's problem). The compute flavour is **FaaS** (Functions as a Service).

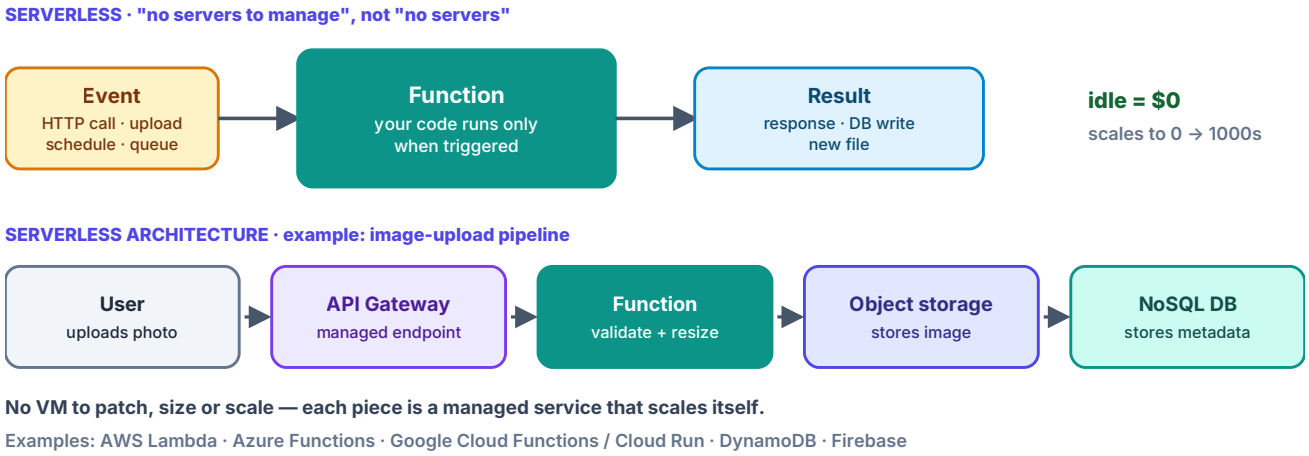


Figure 2.6 — A function triggered by events, and a full serverless architecture

Advantages

- **No server management**.
- **Auto-scaling** from zero to thousands.
- **Pay per use**, nothing when idle.
- Fast to build and ship.
- Naturally event-driven.

Limitations

- **Cold start** — first call after idle is slower.
- **Time limits** (e.g. 15 min on Lambda) — not for long jobs.
- Stateless: keep state in a DB or storage.
- Harder local testing and debugging.
- Costly at very high, constant load.

AWS LAMBDA · PYTHON

```python
import json

def lambda_handler(event, context):
    # triggered when a file lands in the bucket
    for rec in event['Records']:
        key = rec['s3']['object']['key']
        print(f'new file: {key}')
    return {'statusCode': 200, 'body': json.dumps('ok')}
```

DATA ENGINEERING TIE-IN

A classic pipeline: a file lands in object storage → an event triggers a function → it validates the file and starts an ETL job or loads a table. Serverless also covers **serverless databases and warehouses** (BigQuery, Aurora Serverless) where you do not size clusters.

COMMON TRAP

"Serverless" is not "no servers" and not always cheaper. For constant heavy traffic, always-on containers or VMs can cost less.

2.7 Scalability vs Elasticity

★★★

IN ONE LINE

Scalability is the ability to handle growth; elasticity is doing so automatically, in both directions.

Scalability is the ability of a system to handle **growth** by adding resources. **Elasticity** is the ability to **automatically add and remove** resources as demand changes, so you never pay for idle capacity. Elasticity = scalability + automation + shrinking back down.

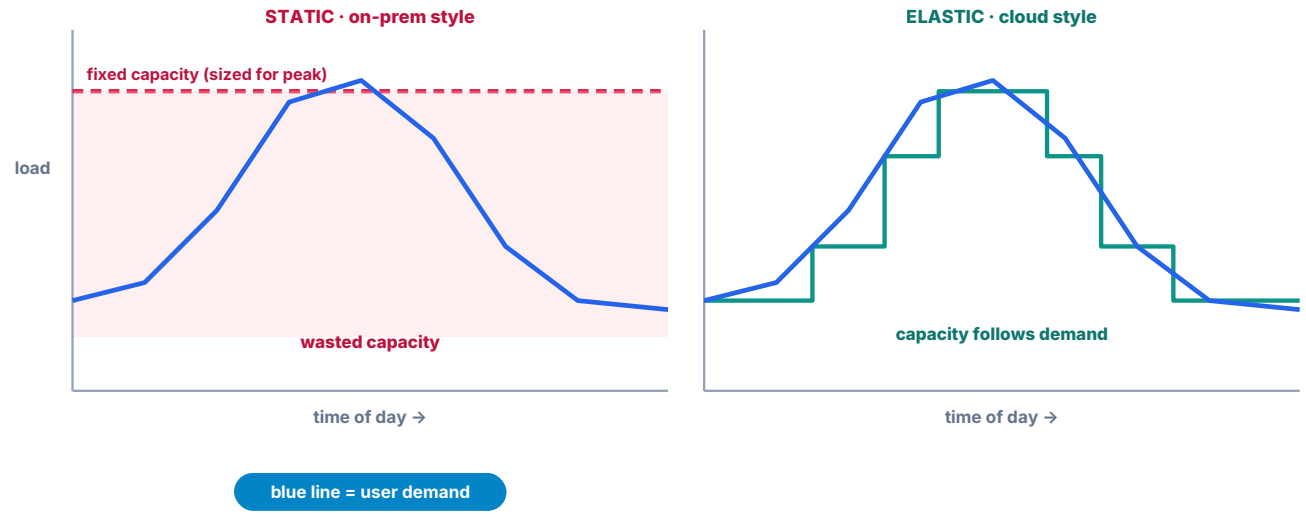


Figure 2.7 — Capacity vs demand over a day

	Scalability	Elasticity
Question it answers	"Can the system handle more load?"	"Can it grow *and shrink* automatically?"
Direction	Mostly growth over the long term	Up and down in near real time
Mechanism	Design: stateless services, partitioned data	Automation: auto scaling, serverless
Time scale	Planned growth, months / years	Minutes / seconds, spikes and quiet hours
Example	Architecture can support 10x users after redesign	Online shop adds servers on sale day, removes them after

REAL-WORLD ANALOGY

A restaurant that can add more tables when it expands (scalable) vs one that brings in extra staff during the dinner rush and sends them home after (elastic).

2.8 Vertical vs Horizontal Scaling

★★★

IN ONE LINE

Scale up by making a machine bigger; scale out by adding more machines.

The two ways to add capacity. **Vertical scaling (scale up)** makes one machine bigger. **Horizontal scaling (scale out)** adds more machines and spreads the work across them.

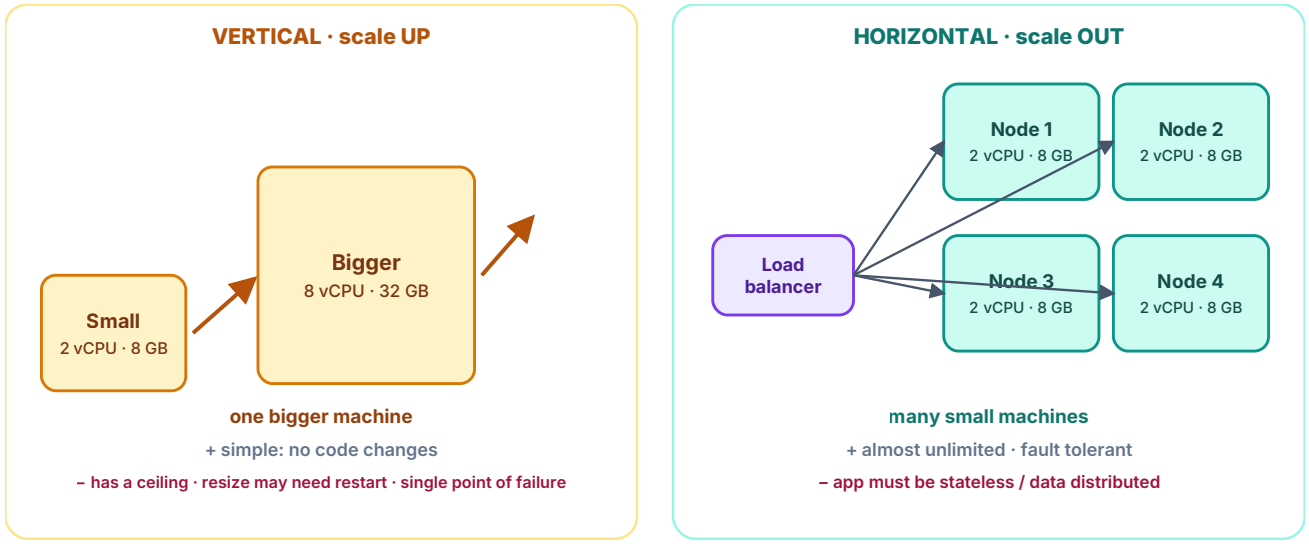


Figure 2.8 — Scale up (bigger) vs scale out (more)

	Vertical (up / down)	Horizontal (out / in)
How	Resize to a bigger instance type	Add or remove instances
Limit	Biggest machine available	Practically none
Downtime	Often needs a restart	None: add nodes while running
Fault tolerance	Single point of failure	Survives node failures
Complexity	Simple, no app changes	Needs load balancer + stateless design
Cost curve	Gets expensive at the top end	Roughly linear, uses commodity machines
Typical use	Relational databases, quick fixes	Web servers, Spark workers, microservices, Kafka brokers

REMEMBER

Scaling **up/down** = change the size of one machine. Scaling **out/in** = change the number of machines. Cloud designs prefer **out** because it is elastic and resilient.

DATA ENGINEERING TIE-IN

Big-data engines such as Spark, Kafka and Hadoop are built to scale **horizontally** — that is the whole idea of distributed processing. Traditional relational databases often scale vertically first, then use read replicas or sharding.

2.9 Auto Scaling

★★★

IN ONE LINE

Automatically adding and removing capacity based on demand, within limits you define.

Auto Scaling is the cloud feature that implements elasticity: it watches a metric (CPU, request count, queue length) and **automatically adds or removes instances** within limits you set. Examples: AWS Auto Scaling groups, Azure VM Scale Sets, GCP Managed Instance Groups.

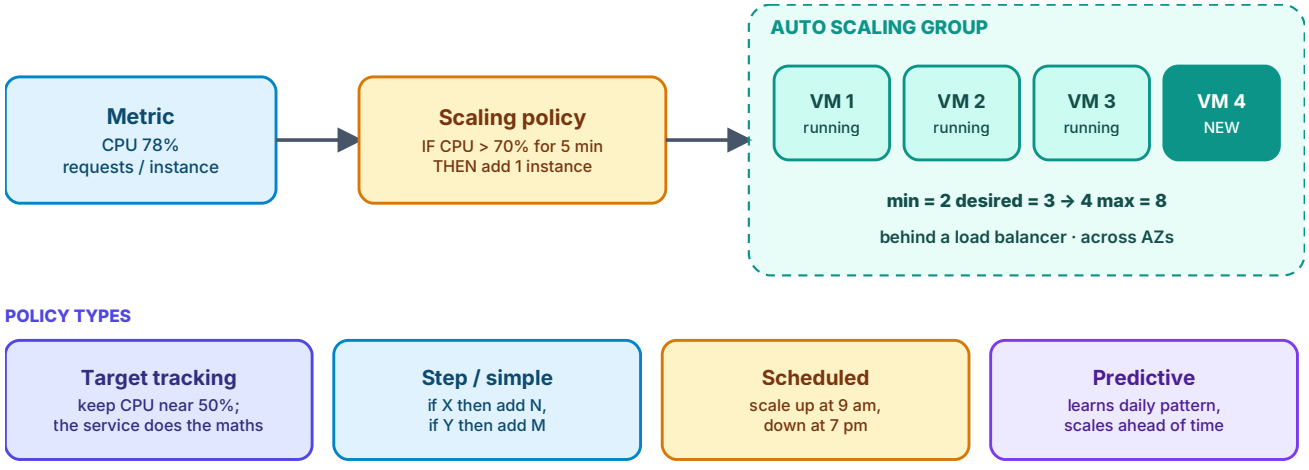


Figure 2.9 — Metric → policy → scale out, with min / desired / max

Setting	Meaning
Min	Never go below this many instances (guarantees baseline availability).
Desired	How many should be running right now; the group moves it up and down.
Max	Hard cap, protects you from runaway cost.
Health check	Unhealthy instances are terminated and replaced automatically (self-healing).
Cooldown	Wait time after a scaling action before the next one, to avoid thrashing.
Launch template	Describes the VM to launch: image, size, security group, startup script.

INTERVIEW ANGLE

Auto scaling needs **stateless instances** (no data stored only on the VM) and a **load balancer** to spread traffic over the changing set of VMs. Putting the group across **multiple Availability Zones** gives scaling *and* high availability at once.

COMMON TRAP

Always set a sensible **max** and a budget alert. A bug or traffic attack that triggers endless scale-out becomes a very large bill.